voorbeeld examen

Academiejaar

Opleiding	Toegepaste informatica	**Jaar**	1 TIN
Opleidingsonderdeel	Web Development II		
Lesgever(s)			

Behaald resultaat
/

☐ Tijdens het examen mogen **geen** hulpmiddelen gebruikt worden.

☒ Tijdens het examen mogen deze hulpmiddelen gebruikt worden:

1. Browser
   a. https://exam.hogent.be
   b. https://developer.mozilla.org
2. Pdf reader om alle gepubliceerde pdf's te kunnen raadplegen
3. Visual Studio Code met enkel en alleen Live Server, Todo Tree, vscode-pdf en Prettier extensies
4. Uitgeprint materiaal mits dit gebundeld is
5. Digitaal materiaal (eigen notities, boeken, modeloplossingen oefeningen, …)

Algemene richtlijnen

Je mag geen enkele vorm van communicatie – noch offline noch online – gebruiken tijdens dit examen, tenzij anders aangegeven in de exameninstructies. Mobiele telefoons, smartwatches en dergelijke moeten uitgeschakeld zijn (niet op stil, trillen, vliegtuigstand, …). Ze mogen tijdens het examen ook niet gebruikt worden om de tijd te raadplegen. Het niet volgen van de gedragscode wordt gesanctioneerd als examenfraude.

HO GENT

HO
GENT

Oefening 01 [50 punten]

Vraag 01

Deel A

Implementeer de functie **`addName`**. De functie moet `newName` achteraan toevoegen aan de array `allNames` met een correcte casing: de toegevoegde naam start met een hoofdletter, gevolgd door kleine letters.

Na functie-aanroepen voor het toevoegen van `'aNnIe'` en `'EliSA'` aan de array `names` ziet de array er als volgt uit:

```
['Jude', 'Jude', 'Lana', 'Drew', 'Jude', 'Jude', 'Jude', 'Lana', 'Annie', 'Elisa']
```

Deel B

De gegeven functie **`isPopularName`** retourneert `true` indien `name` meer dan **1** keer voorkomt in de array `allNames`.

Pas de functie aan zodat ze kan aangeroepen worden met een **optionele derde parameter** genaamd **`popularityLevel`**. Dit zal een getal zijn.

Indien een waarde voor de optionele parameter `popularityLevel` wordt aangeleverd moet de functie `true` retourneren indien `name` meer dan `popularityLevel` keer voorkomt in de array. Indien de functie wordt aangeroepen zonder `popularityLevel` blijft het gedrag zoals voorheen.

In de console zie je voor dit deel volgende uitvoer:

```
isPopularName('Jude', names, 3) -> true
isPopularName('Jude', names, 5) -> false
isPopularName('Lana', names) -> true
isPopularName('Drew', names) -> false
isPopularName('Drew', names, 0) -> true
```

Deel C

Geef een invulling aan de functie **`giveNamesWithoutRepetition`**. De functie retourneert een nieuwe array `allNames` waarbij opeenvolgende reeksen van eenzelfde naam zijn gereduceerd tot 1 keer de naam.

In de console zie je het resultaat van een aanroep met de array

`['Jude', 'Jude', 'Lana', 'Drew', 'Drew', 'Jude', 'Jude', 'Jude', 'Lana']`

en dit ziet er als volgt uit:

```
['Jude', 'Lana', 'Drew', 'Jude', 'Lana']
```

HO GENT

Vraag 02

Vervolledig de gegeven klasse **`Trekking`**.

Deel A

Voeg een publieke setter `getallen` toe voor de property `#getallen`. Zorg dat de aangeleverde getallen gecontroleerd worden op hun geldigheid door een aanroep naar de methode `#controleerGetallen` te doen alvorens deze toe te kennen aan `#getallen`.

Deel B

Vervolledig de **`constructor`**. Maak gebruik van de setter `getallen` om aan de array `#getallen` de aangeleverde array `getallen` toe te kennen.

Deel C

Vervolledig de methode **`#controleerGetallen`**

- Pas de string `errorMessage` aan zodat gebruik wordt gemaakt van de static properties van de klasse in plaats van de hard gecodeerde getallen 6, 1 en 42. Maak gebruik van een template literal (i.e. backticks gebruiken).
- De methode moet een `Error` (maak gebruik van `errormessage`) werpen indien de aangeleverde array `getallen` niet **exact 6 verschillende gehele getallen** uit het **interval [1, 42]** bevat. Je mag er van uit gaan dat de array `getallen` enkel gehele getallen bevat, controleer dus nog, gebruik makend van de static properties van de klasse,
  - of de array **exact 6 verschillende** getallen bevat
  - of elk getal in het interval **[1, 42]** ligt

De voorbeelduitvoer ziet er als volgt uit:

```
De getallen van de trekking zijn 1,2,3,4,5,6
Enkel 6 verschillende gehele getallen in het interval [1, 42] zijn toegelaten!
Enkel 6 verschillende gehele getallen in het interval [1, 42] zijn toegelaten!
Enkel 6 verschillende gehele getallen in het interval [1, 42] zijn toegelaten!
Enkel 6 verschillende gehele getallen in het interval [1, 42] zijn toegelaten!
Enkel 6 verschillende gehele getallen in het interval [1, 42] zijn toegelaten!
```

Vraag 03

Deel A

Voeg aan de Map **`planets`** een entry toe voor de planeet Neptune. Deze planeet heeft `16` manen (`nrOfmoons`) en staat op `30.06` Au van de zon (`distanceFromSun`).

Deel B

Wijzig de afstand van de zon voor de planeet Venus naar `0.72`

Deel C

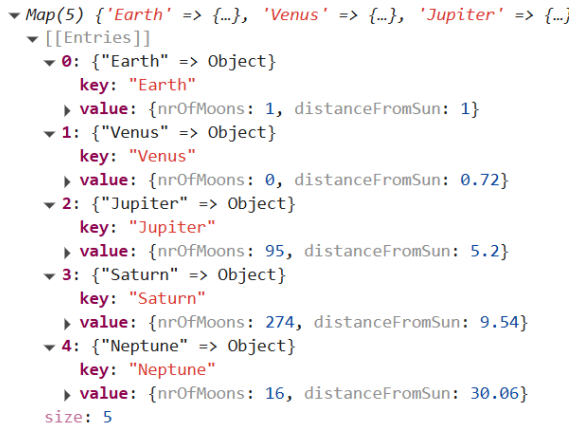
Verwijder de planeet Pluto uit de map `planets`.

Deel D

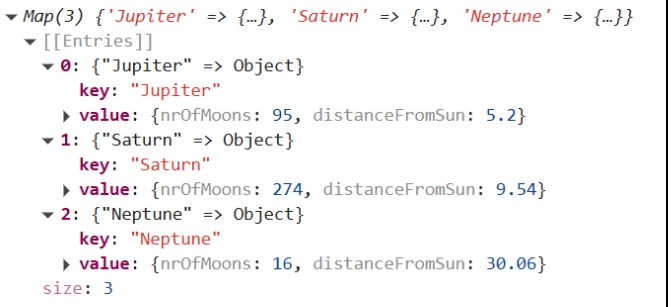
Ken aan de variabele `planetsFarAway` een nieuwe map toe die, vertrekkend van de planeten in de variabele `planets`, enkel die planeten bevat die een afstand tot de zon (`distanceToSun`) hebben die groter is dan 1.

Nadat je dit deel hebt opgelost zien de twee variabelen `planets` en `planetsFarAway` er als volgt uit:

planets



planetsFarAway



HO GENT

Vraag 04

Deel A

We moeten instanties van de klasse **`ComplexObject`** kunnen wegschrijven naar de **`sessionStorage`**. Voeg de **`toJSON`** methode toe aan de klasse `ComplexObject` om dit mogelijk te maken.

Deel B

Schrijf `originalObject` weg naar de `sessionStorage`. Kies zelf een waarde voor de sleutel. Je hoeft geen error handling te doen.

Deel C

Lees de weggeschreven gegevens uit de `sessionStorage` terug uit, en maak aan de hand van deze gegevens en de constructor van `ComplexObject` de variabele `duplicateObject`. Als je dit correct doet zal deze instantie van `ComplexObject` gelijk zijn aan `originalObject` (i.e. de gegeven `equals` methode zal `true` opleveren).

Je krijgt volgende uitvoer:

```
originalObject en duplicateObject zijn gelijk!
```

```
ComplexObject i
  #date: Fri Jul 30 2032 00:00:00 GMT+0200 (Midden-Europese zomertijd)
  #tags: Array(3)
    0: "t1"
    1: "t2"
    2: 99
    length: 3
    [[Prototype]]: Array(0)
  #title: "abc"
```

```
ComplexObject i
  #date: Fri Jul 30 2032 00:00:00 GMT+0200 (Midden-Europese zomertijd)
  #tags: Array(3)
    0: "t1"
    1: "t2"
    2: 99
    length: 3
    [[Prototype]]: Array(0)
  #title: "abc"
```

Vraag 05

De array `songs` bevat objecten met gegevens over liedjes.

Deel A

Ken aan de variabele `shortSongs` een array toe die enkel die liedjes uit de array `songs` bevat die minder dan `250` seconden (`duration`) duren.

Voorbeelduitvoer:

```
(2) [{…}, {…}]
  0: {title: 'M for music', duration: 204, genres: Array(1)}
  1: {title: 'Music from Mars', duration: 242, genres: Array(2)}
  length: 2
```

Deel B

Ken aan de variabele `sortedSongs` een array toe met alle liedjes **gesorteerd op het aantal genres** waartoe ze behoren. Voor gelijk aantal genres sorteer je de liedjes **alfabetisch op** `title`.

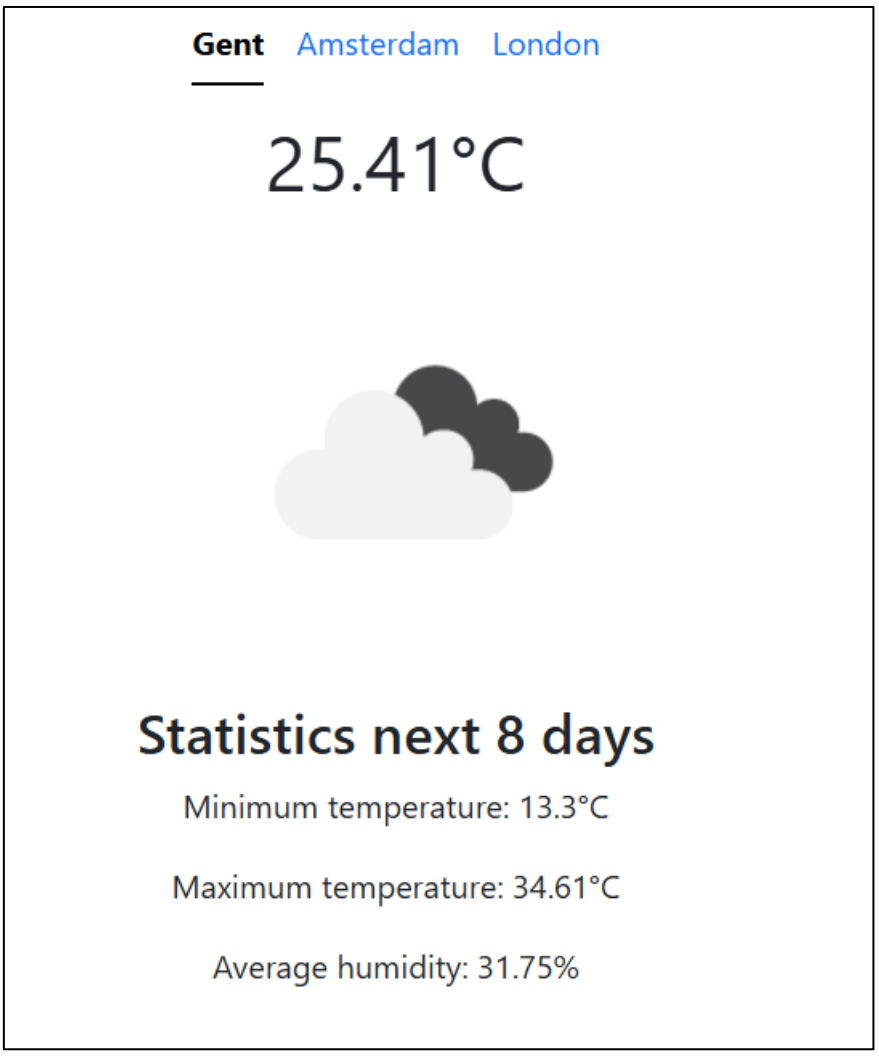
Voorbeelduitvoer:

```
(5) [{…}, {…}, {…}, {…}, {…}]
  0: {title: 'Breathing music', duration: 302, genres: Array(0)}
  1: {title: 'Life is music', duration: 274, genres: Array(1)}
  2: {title: 'M for music', duration: 204, genres: Array(1)}
  3: {title: 'Music from Mars', duration: 242, genres: Array(2)}
  4: {title: 'Love that music', duration: 265, genres: Array(3)}
  length: 5
```

Oefening 02 [50 punten]

In deze oefening maken we een webpagina die het **huidige weer** en een w**eersvoorspelling voor de komende dagen** toont op basis van gegevens van de OpenWeather One Call API 3.0. Op de webpagina kan de gebruiker een locatie kiezen uit een reeks van voorgedefinieerde locaties. Initieel wordt Gent gekozen. Van de gekozen locatie wordt volgende informatie weergegeven

- De huidige temperatuur en een bijpassend icoon
- De statistieken van de komende 8 dagen: de minimum temperatuur, de maximum temperatuur en de gemiddelde vochtigheidsgraad

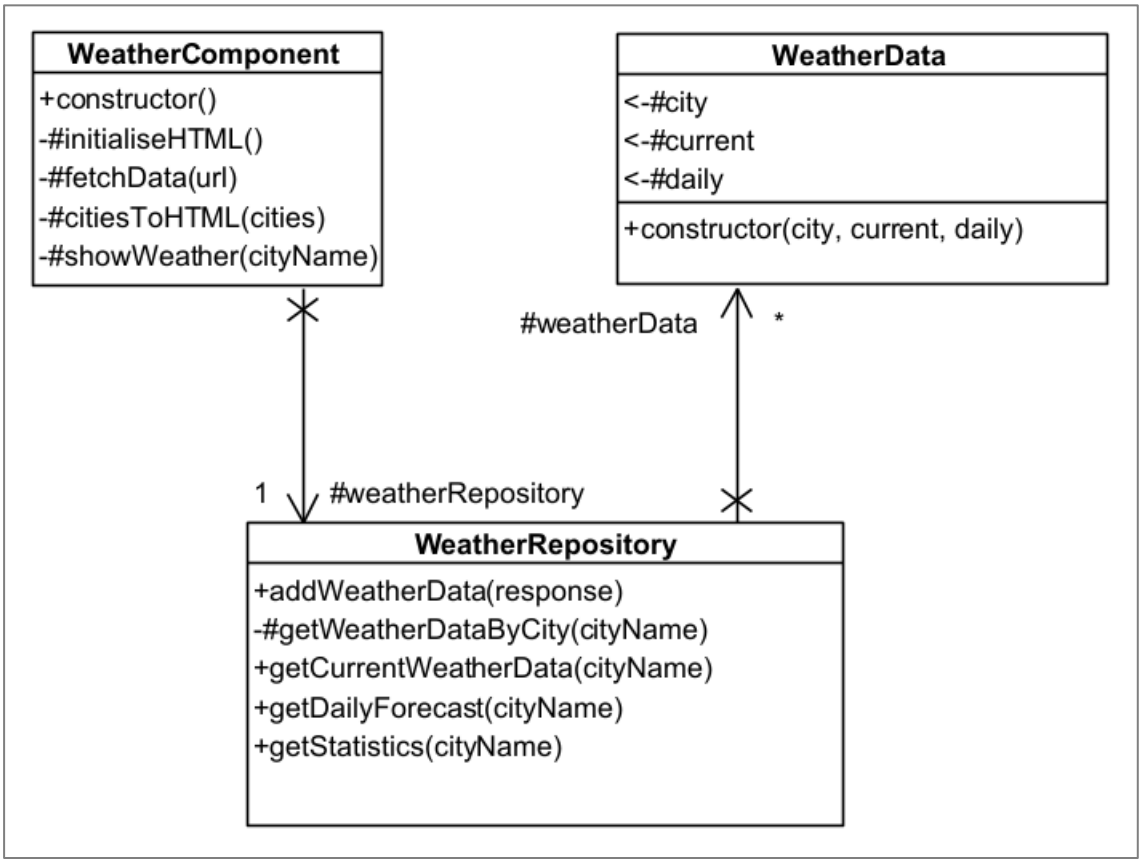


De **JSON-data** die de OpenWeather One Call API 3.0 retourneert kan je terugvinden in het bestand **weatherGentAmsterdamLonden.json** in de **data-json** folder. Dit bevat de weersvoorspellingen opgehaald op 18/6/2025. De data bestaat uit een array die voor *elke stad een object met de huidige weergegevens en de voorspellingen* bevat. Per stad heb je

- algemene gegevens zoals ‘city’, ‘country’,…

- ‘current’: het huidige weer. ‘dt’ is de datum (unix formaat), temp (de temperatuur), ‘weather’ bevat het icon (‘icon’) en de beschrijving (‘description’). Merk op dat de Unix datum in de code (class WeatherData) met 1000 wordt vermenigvuldigd om een datum-tijd te bekomen.
- ‘daily’: de weersvoorspellingen voor de 8 komende dagen. Per dag heb je
  - ‘dt’: de datum (unix formaat)
  - ‘temp’: de temperatuursgegevens met ‘min’, de minimum temperatuur en ‘max’ de maximum temperatuur
  - ‘weather’ bevat het icon(‘icon’) en de beschrijving (‘description’)

Het klassendiagram:



De klasse **WeatherData** is reeds grotendeels geïmplementeerd en ook de klassen **WeatherRepository** en **WeatherComponent** zijn reeds gedeeltelijk geïmplementeerd. Implementeer deel a, b en c om tot een werkende pagina te komen. Baseer je op de UML, de uitleg en de schermafbeeldingen.

Er is ook code voorzien om de klassen **WeatherData** en **WeatherRepository** te testen. Zie verder in deze opgave.

Je mag niets wijzigen in **index.html**.

Deel a – De klasse WeatherData (4 ptn)

Deze klasse bevat de weersgegevens voor een stad:

- #city: de naam van de stad
- #current: de huidige weersituatie

- #daily: een array met de 8 voorspellingen

Implementeer in de **constructor** het ophalen van het icon en de description voor het huidige weer.

Je kan je code testen door het bestand **indexTest.html** te openen dat je vindt in de map **test**. De console geeft volgende output weer voor Weather gent



Deel b – De klasse WeatherRepository (12 ptn)

De klasse bevat de **array #weatherData** met WeatherData-instanties (1 per stad). Vervolledig volgende code.

- De methode **addWeatherData(jsonData)**
  - jsonData bevat een array met literal objects opgebouwd volgens de gegevens in weatherGentAmsterdamLondon.json. Voor elk literal object moet in de array #weatherdata een Weatherdata object komen (dus een instantie voor elke stad).
- De methode **#getWeatherDataByCity(cityName)**
  - Retourneert de WeatherData instantie van de opgegeven cityName
- De methode **getStatistics(cityName)**
  - Retourneert de voorspellingen voor de opgegeven stad. Dit is een object met de properties
    - minTemperature: de laagste temperatuur uit de voorspellingen
    - maxTemperature: de hoogste temperatuur uit de voorspellingen
    - averageHumidity: de gemiddelde vochtigheidsgraad uit de voorspellingen

Je kan je code testen door het bestand **indexTest.html** te openen dat je vindt in de map **test**. De console geeft onderstaande uitvoer weer:



Deel c – De klasse WeatherComponent (34 ptn)

- De methode **#fetchData(url)**
  - Gebruik de **fetch API** om de data op te halen uit het bestand met de gegeven **url**. Maak gebruik van **async/await**.

  Indien je de data niet kan ophalen via de fetch-api kan je beslissen gebruik te maken van de array hulplijnWeather die onder de klasse WeatherComponent is gedeclareerd. Als je deze array gebruikt om de repository op te vullen kan je de rest van je werk wel in uitvoering zien. Uiteraard krijg je dan geen punten voor het asynchroon ophalen en verwerken van de data!

  - Voeg de opgehaalde data toe aan **#weatherRepository**.
  - Werp een **Error** met een gepaste foutboodschap indien de data niet correct kan worden opgehaald en geef een alert met de foutboodschap bij het opvangen van de exceptions.

- De methode **#initialiseHTML()**
  - Roep de methode **#fetchData** aan. Geef hierbij de gepaste url door, de data moet worden opgehaald uit het **.json** bestand die in de map **json-data** zit. De naam van het bestand dient dynamisch te worden opgebouwd. Hiervoor concateneer je de namen van de steden in de **#cities** array
  - Roep de methode **#citiesToHTML** aan. Geef de array **#cities** door.

HO GENT

- Roep de methode **#showWeather** aan. Geef de naam van de eerste stad in de array **#cities** door.

- De methode **#citiesToHTML (cities)**



- Vul het gegeven ul-element met id 'cities' op met een li-element voor elke stad uit de array cities. De steden zijn hyperlinks. Zie HTML hieronder.
- **Event handling**: wanneer de gebruiker klikt op een stad, worden de weersvoorspellingen van deze stad getoond.
  - De methode **#showWeather** wordt aangeroepen voor deze stad.
  - De link voor deze stad wordt de actieve link (class "active"). De reeds actieve links worden inactief.
    - Voorbeeld HTML voor Gent:

```
<ul class="nav justify-content-center nav-underline" id="cities">
  <li class="nav-item"><a class="nav-link active" href="#">Gent</a></li>
  <li class="nav-item"><a class="nav-link" href="#">Amsterdam</a></li>
  <li class="nav-item"><a class="nav-link" href="#">London</a></li>
</ul>
```

- De methode **#showWeather(cityname)**



- Het huidige weer voor deze stad wordt weergegeven in de div met id **'weather-info'**. De HTML voor Gent is

```
<div id="weather-info" class="mx-auto p-2" style="width: 200px">
  <p class="fs-1 text-center">25.41°C</p>
  <p><img width="200" height="200" src="./images/04d.png" alt="broken clouds"></p>
</div>
```

- De **statistieken** voor deze stad worden weergegeven in de div met id **'statistics'**. Voorbeeld HTML voor Gent:

```
<div id="statistics" class="text-center">
  <h3>Statistics next 8 days</h3>
  <p>Minimum temperature: 13.3°C</p>
  <p>Maximum temperature: 34.61°C</p>
  <p>Average humidity: 31.75%</p>
</div>
```